

Лабораторная работа №10.

Все поля должны быть приватными! Все поля необходимо инициализировать в конструкторе! В свойствах и методах перед обращением к ссылочным полям проверять их на null. Вам нужно решить задания в точности так, как указано в условии. Для выполнения задания вы можете использовать дополнительные приватные поля, свойства или методы.

Для выполнения работы требуется скопировать папку **Lab9** из проекта девятой лабораторной работы в проект десятой лабораторной. Проекты **Lab10** и **Lab10Test** должны иметь ссылку на проект **Lab9**. Рекомендуется удалить из папки **Lab9** подпапки **bin** и **obj** после переноса.

Для выполнения задания вы можете использовать дополнительные приватные или защищенные поля, свойства, методы, структуры и классы. Они не должны быть видны внешним программам!

Если все тесты (автоматические и ручные) пройдены успешно, Вы можете отправить лабораторную на заключительную проверку на **GitHub**. Более подробная инструкция описана в задании на **Moodle**.

Если Ваша работа принята, вы можете прорешать номера из других лиг в качестве подготовки к контрольной. Не обязательно все, а только те, которые не похожи на то, что было в Вашей лиге. Рекомендуется так сделать для тех, кто не имеет опыта в программировании и был определен в белую лигу, потому что для успешного освоения курса Вам нужно научиться решать задачи уровня зеленой и частично синей лиги.

В работе разрешены методы следующих классов: *Console*, *Math*, *Random*, *Array*, *Linq*, *Dictionary*, *List*, *JsonSerializer*, *XmlSerializer*. Также разрешено подключение библиотек для работы с JSON/XML сериализацией.

Задания общие (для всех лиг).

Создать интерфейс **IFileManager** с набором строковых свойств **FolderPath**, **FileName**, **FileExtension**, **FullPath**. И методов **SelectFolder**, **ChangeFileName**, **ChangeFileFormat**, принимающих строковое поле.

Создать интерфейс **IFileLifeController** с набором методов **CreateFile**, **DeleteFile**, не требующих входных параметров, и методов **EditFile** и **ChangeFileExtension**, принимающих строковое поле.

Создать абстрактный класс **MyFileManager**, реализующий интерфейсы **IFileManager** и **IFileLifeController**.

Один конструктор класса должен принимать строковое имя для свойства Name данного класса. Вторым конструктором – имя, название папки, название файла и необязательный параметр для расширения файла со значением “txt” по умолчанию.

Создать свойства интерфейса **IFileManager**, чтобы они возвращали значение переданных в конструктор данных. Свойство **FullPath** должно возвращать полный путь к файлу: путь к папке, имя файла и расширение файла.

Создать методы интерфейса **IFileManager**, чтобы они меняли значения соответствующих им свойств.

Создать методы интерфейса **IFileLifeController**.

Метод **CreateFile** должен создавать файл по пути **FullPath**, а также недостающие папки, если это требуется.

Метод **DeleteFile** должен удалять файл по пути **FullPath**, если он имеется.

Метод **EditFile** должен заменять содержимое файла на переданное.

Метод **ChangeFileExtension** должен менять расширение файла с сохранением содержимого, если это возможно. Старый файл должен удаляться.

Задания белой лиги.

Создать интерфейс **IWhiteSerializer** с методом **Deserialize**, не принимающих входных параметров, но возвращающего тип **Lab9.White.White**, и метода **Serialize**, принимающего объект этого типа данных.

Создать папку **White**. В ней создать файлы **White**, **WhiteFileManager**, **WhiteJsonFileManager**, **WhiteTxtFileManager**.

Файл **White**

Создать класс **White**, предназначенный для хранения и управления набором объектов типа **Lab9.White.White**, а также для работы с файловым менеджером типа **WhiteFileManager**.

Создать публичные свойства только для чтения, возвращающие текущего менеджера **Manager** и массив задач **Tasks**.

Один конструктор должен принимать массив задач и инициализировать внутреннее хранилище, при отсутствии переданных данных должен создаваться пустой массив. Второй конструктор должен принимать менеджер файлов и необязательный массив задач. Третий конструктор должен принимать массив задач и менеджер файлов. Во всех случаях переданный массив должен копироваться во внутреннее хранилище или же должен создаваться пустой массив, если ничего не было передано или был передан null.

Создать метод **Add** для добавления одного объекта типа **Lab9.White.White** в коллекцию с увеличением размера массива. Создать перегруженный метод **Add** для добавления массива объектов с последовательным добавлением каждого элемента.

Создать метод **Remove**, который удаляет переданный объект из коллекции. Создать метод **Clear**, который очищает коллекцию задач и удаляет директорию, связанную с текущим файловым менеджером.

Создать метод **SaveTasks**, который последовательно сохраняет все задачи в файлы с использованием текущего файлового менеджера. Имя файла должно изменяться для каждой задачи.

Создать метод **LoadTasks**, который последовательно загружает задачи из файлов с использованием текущего файлового менеджера и обновляет элементы массива.

Создать метод **ChangeManager**, который заменяет текущего менеджера на нового, создаёт папку на основе имени менеджера и устанавливает её в качестве рабочей папки для нового менеджера.

Файл **WhiteFileManager**

Создать абстрактный класс **WhiteFileManager**, наследующий класс **MyFileManager** и реализующий интерфейс **IWhiteSerializer**.

Конструкторы должны быть идентичны конструкторам базового класса **MyFileManager**.

Методы **Serialize** и **Deserialize** должны быть объявлены как абстрактные.

Переопределить методы **EditFile** и **ChangeFileExtension**, сохранив базовую логику работы с файлом, используя свойство **FullPath** и проверки корректности пути.

Файл **WhiteJsonFileManager**

Создать класс **WhiteJsonFileManager**, наследующий класс **WhiteFileManager**.

Конструкторы должны быть идентичны конструкторам базового класса **WhiteFileManager**.

Переопределить метод **EditFile** таким образом, чтобы выполнялась десериализация объекта из файла по пути **FullPath** с помощью метода **Deserialize**, затем вызывался метод **ChangeText** у полученного объекта, после чего объект сериализовывался обратно в файл по тому же пути с помощью метода **Serialize**.

Переопределить метод **ChangeFileExtension** таким образом, чтобы расширение файла изменялось только на «json» с использованием метода **ChangeFileFormat**.

Создать метод **Serialize** таким образом, чтобы он преобразовывал объект типа **Lab9.White.White** в JSON-представление, дополнял его информацией о типе объекта и сохранял результат в файл по пути **FullPath**.

Создать метод **Deserialize** таким образом, чтобы он считывал содержимое файла по пути **FullPath**, определял тип объекта на основе сохранённых данных и создавал соответствующий объект типа **Lab9.White.White** или его наследников с последующей инициализацией данных.

Файл **WhiteTxtFileManager**

Создать класс **WhiteTxtFileManager**, наследующий класс **WhiteFileManager**.

Конструкторы должны быть идентичны конструкторам базового класса **WhiteFileManager**.

Переопределить метод **EditFile** таким образом, чтобы выполнялась десериализация объекта из файла по пути **FullPath** с помощью метода **Deserialize**, затем вызывался метод **ChangeText** у объекта, после чего объект сериализовывался обратно в файл с помощью метода **Serialize**.

Переопределить метод **ChangeFileExtension** таким образом, чтобы расширение файла изменялось только на «txt» с использованием метода **ChangeFileFormat**.

Создать метод **Serialize** таким образом, чтобы он преобразовывал объект типа **Lab9.White.White** в текстовое представление в виде набора пар ключ-значение и сохранял результат в файл по пути **FullPath**. Структура данных должна обеспечивать возможность последующего восстановления объекта при помощи метода **Deserialize**.

Создать метод **Deserialize** таким образом, чтобы он считывал содержимое файла по пути **FullPath**, восстанавливал набор данных из текстового представления, определял тип объекта и создавал

соответствующий объект типа **Lab9.White.White** или его наследников с последующей инициализацией данных.

Задания зеленой лиги.

Создать интерфейс **ISerializer** с обобщенным (параметрическим) методом **Deserialize**, не принимающих входных параметров, но возвращающего объект типа **T : Lab9.Green.Green**, и обобщенного метода **Serialize**, принимающего объект этого типа данных.

Создать папку **Green**. В ней создать файлы **Green**, **GreenFileManager**, **GreenJsonFileManager**, **GreenTxtFileManager**.

Файл **Green**

Создать класс **Green**, предназначенный для хранения и управления набором объектов типа **Lab9.Green.Green**, а также для работы с файловым менеджером типа **GreenFileManager**.

Создать публичные свойства только для чтения, возвращающие текущего менеджера **Manager** и массив задач **Tasks** типа **Lab9.Green.Green**.

Один конструктор должен принимать массив задач и инициализировать внутреннее хранилище, при отсутствии переданных данных должен создаваться пустой массив. Второй конструктор должен принимать менеджер файлов и необязательный массив задач. Третий конструктор должен принимать массив задач и менеджер файлов. Во всех случаях переданный массив должен копироваться во внутреннее хранилище или же должен создаваться пустой массив, если ничего не было передано или был передан null.

Создать метод **Add** для добавления одного объекта типа **Lab9.Green.Green** в коллекцию с увеличением размера массива. Создать перегруженный метод **Add** для добавления массива объектов с последовательным добавлением каждого элемента.

Создать метод **Remove**, который удаляет переданный объект из коллекции. Создать метод **Clear**, который очищает коллекцию задач и удаляет директорию, связанную с текущим файловым менеджером через свойство **FolderPath**.

Создать метод **SaveTasks**, который последовательно сохраняет все задачи в файлы с использованием текущего файлового менеджера. Имя файла должно изменяться для каждой задачи с использованием метода **ChangeFileName**, а сохранение должно выполняться через обобщенный метод **Serialize**.

Создать метод **LoadTasks**, который последовательно загружает задачи из файлов с использованием текущего файлового менеджера с помощью обобщенного метода **Deserialize** и обновляет элементы массива.

Создать метод **ChangeManager**, который заменяет текущего менеджера на нового, создаёт папку на основе свойства **Name** менеджера (если она отсутствует) и устанавливает её в качестве рабочей папки для нового менеджера с помощью метода **SelectFolder**.

Файл **GreenFileManager**

Создать абстрактный класс **GreenFileManager**, наследующий класс **MyFileManager** и реализующий интерфейс **ISerializer**.

Конструкторы должны быть идентичны конструкторам базового класса **MyFileManager**.

Переопределить методы **EditFile** и **ChangeFileExtension**, сохранив базовую логику работы с файлом, используя свойство **FullPath** и проверки корректности пути.

Методы **Serialize** и **Deserialize** должны быть объявлены как абстрактные и обобщённые. Метод **Serialize** должен принимать объект обобщённого типа, ограниченного типом **Lab9.Green.Green**. Метод **Deserialize** должен возвращать объект обобщённого типа с тем же ограничением.

Файл **GreenJsonFileManager**

Создать класс **GreenJsonFileManager**, наследующий класс **GreenFileManager**.

Конструкторы должны быть идентичны конструкторам базового класса **GreenFileManager**.

Переопределить метод **EditFile** таким образом, чтобы выполнялась десериализация объекта из файла по пути **FullPath** с помощью обобщённого метода **Deserialize**, затем вызывался метод **ChangeText** у полученного объекта, после чего объект сериализовывался обратно в файл по тому же пути с помощью обобщённого метода **Serialize**.

Переопределить метод **ChangeFileExtension** таким образом, чтобы расширение файла изменялось только на «json» с использованием метода **ChangeFileFormat**.

Создать обобщённый метод **Serialize** таким образом, чтобы он преобразовывал объект обобщённого типа, наследуемого от **Lab9.Green.Green**, в JSON-представление, дополнял его информацией о типе объекта и сохранял результат в файл по пути **FullPath**.

Создать обобщённый метод **Deserialize** таким образом, чтобы он считывал содержимое файла по пути **FullPath**, определял тип объекта на основе сохранённых данных и создавал соответствующий объект обобщённого типа, наследуемого от **Lab9.Green.Green**, с последующей инициализацией данных.

Файл **GreenTxtFileManager**

Создать класс **GreenTxtFileManager**, наследующий класс **GreenFileManager**.

Конструкторы должны быть идентичны конструкторам базового класса **GreenFileManager**.

Переопределить метод **EditFile** таким образом, чтобы выполнялась десериализация объекта из файла по пути **FullPath** с помощью обобщённого метода **Deserialize**, затем вызывался метод **ChangeText** у объекта, после чего объект сериализовывался обратно в файл с помощью обобщённого метода **Serialize**.

Переопределить метод **ChangeFileExtension** таким образом, чтобы расширение файла изменялось только на «txt» с использованием метода **ChangeFileFormat**.

Создать обобщённый метод **Serialize** таким образом, чтобы он преобразовывал объект обобщённого типа, наследуемого от **Lab9.Green.Green**, в текстовое представление в виде набора пар ключ-значение и сохранял результат в файл по пути **FullPath**. Структура данных должна обеспечивать возможность последующего восстановления объекта при помощи метода **Deserialize**.

Создать обобщённый метод **Deserialize** таким образом, чтобы он считывал содержимое файла по пути **FullPath**, восстанавливал набор данных из текстового представления, определял тип объекта и создавал соответствующий объект обобщённого типа, наследуемого от **Lab9.Green.Green**, с последующей инициализацией данных.

Задания синей лиги.

Создать обобщенный (параметрический) интерфейс **ISerializer<T>** : **Lab9.Blue.Blue** с методом **Deserialize**, не принимающих входных параметров, но возвращающего объект типа **T**, и обобщенного метода **Serialize**, принимающего объект этого типа данных.

Создать папку **Blue**. В ней создать файлы **Blue**, **BlueFileManager**, **BlueJsonFileManager**, **BlueTxtFileManager**.

Файл **Blue**

Создать обобщенный класс **Blue**, предназначенный для хранения и управления набором объектов обобщенного типа **T**, где **T** ограничен типом **Lab9.Blue.Blue**, а также для работы с файловым менеджером типа **BlueFileManager<T>**.

Создать публичные свойства только для чтения, возвращающие текущего менеджера **Manager** типа **BlueFileManager<T>** и массив задач **Tasks** типа **T**.

Один конструктор должен принимать массив объектов типа **T** и инициализировать внутреннее хранилище, при отсутствии переданных данных должен создаваться пустой массив. Второй конструктор должен принимать менеджер файлов типа **BlueFileManager<T>** и необязательный массив задач. Третий конструктор должен принимать массив задач и менеджер файлов. Во всех случаях переданный массив должен копироваться во внутреннее хранилище или же должен создаваться пустой массив, если ничего не было передано или был передан **null**.

Создать метод **Add** для добавления одного объекта типа **T** в коллекцию с увеличением размера массива. Создать перегруженный метод **Add** для добавления массива объектов с последовательным добавлением каждого элемента.

Создать метод **Remove**, который удаляет переданный объект из коллекции. Создать метод **Clear**, который очищает коллекцию задач и удаляет директорию, связанную с текущим файловым менеджером через свойство **FolderPath**.

Создать метод **SaveTasks**, который последовательно сохраняет все задачи в файлы с использованием текущего файлового менеджера. Имя файла должно изменяться для каждой задачи с использованием метода **ChangeFileName**, а сохранение должно выполняться с помощью метода **Serialize**.

Создать метод **LoadTasks**, который последовательно загружает задачи из файлов с использованием текущего файлового менеджера с помощью метода **Deserialize** и обновляет элементы массива.

Создать метод **ChangeManager**, который заменяет текущего менеджера на нового, создаёт папку на основе свойства **Name** менеджера (если она отсутствует) и устанавливает её в качестве рабочей папки для нового менеджера с помощью метода **SelectFolder**.

Файл **BlueFileManager**

Создать абстрактный обобщённый класс **BlueFileManager<T>**, наследующий класс **MyFileManager** и реализующий интерфейс **ISerializer<T>**, где параметр типа **T** ограничен типом **Lab9.Blue.Blue**.

Конструкторы должны быть идентичны конструкторам базового класса **MyFileManager**.

Переопределить методы **EditFile** и **ChangeFileExtension**, сохранив базовую логику работы с файлом, используя свойство **FullPath** и проверки корректности пути.

Метод **Serialize** должен быть объявлен как абстрактный и принимать объект типа **T**.

Метод **Deserialize** должен быть объявлен как абстрактный и возвращать объект типа **T**.

Файл **BlueJsonFileManager**

Создать обобщённый класс **BlueJsonFileManager<T>**, наследующий класс **BlueFileManager<T>**, где **T** ограничен типом **Lab9.Blue.Blue**.

Конструкторы должны быть идентичны конструкторам базового класса **BlueFileManager<T>**.

Переопределить метод **EditFile** таким образом, чтобы выполнялась десериализация объекта из файла по пути **FullPath** с помощью метода **Deserialize**, затем вызывался метод **ChangeText** у полученного объекта, после чего объект сериализовывался обратно в файл по тому же пути с помощью метода **Serialize**.

Переопределить метод **ChangeFileExtension** таким образом, чтобы расширение файла изменялось только на «json» с использованием метода **ChangeFileFormat**.

Реализовать метод **Serialize** таким образом, чтобы он преобразовывал объект типа **T** в JSON-представление, дополнял его информацией о типе объекта и сохранял результат в файл по пути **FullPath**.

Реализовать метод **Deserialize** таким образом, чтобы он считывал содержимое файла по пути **FullPath**, определял тип объекта на основе сохранённых данных и создавал соответствующий объект типа **T** с последующей инициализацией данных.

Файл **BlueTxtFileManager**

Создать обобщённый класс **BlueTxtFileManager<T>**, наследующий класс **BlueFileManager<T>**, где **T** ограничен типом **Lab9.Blue.Blue**.

Конструкторы должны быть идентичны конструкторам базового класса **BlueFileManager<T>**.

Переопределить метод **EditFile** таким образом, чтобы выполнялась десериализация объекта из файла по пути **FullPath** с помощью метода **Deserialize**, затем вызывался метод **ChangeText** у объекта, после чего объект сериализовывался обратно в файл с помощью метода **Serialize**.

Переопределить метод **ChangeFileExtension** таким образом, чтобы расширение файла изменялось только на «txt» с использованием метода **ChangeFileFormat**.

Реализовать метод **Serialize** таким образом, чтобы он преобразовывал объект типа **T** в текстовое представление в виде набора пар ключ-значение и сохранял результат в файл по пути **FullPath**. Структура данных должна обеспечивать возможность последующего восстановления объекта при помощи метода **Deserialize**.

Реализовать метод **Deserialize** таким образом, чтобы он считывал содержимое файла по пути **FullPath**, восстанавливал набор данных из текстового представления, определял тип объекта и создавал соответствующий объект типа **T** с последующей инициализацией данных.

Задания фиолетовой лиги.

Создать обобщенный (параметрический) интерфейс **ISerializer<T>** : **Lab9.Purple.Purple** с методом **Deserialize**, не принимающих входных параметров, но возвращающего объект типа **T**, и обобщенного метода **Serialize**, принимающего объект этого типа данных.

Файл Purple

Создать обобщённый класс **Purple**, предназначенный для хранения и управления набором объектов обобщённого типа **T**, где **T** ограничен типом **Lab9.Purple.Purple**, а также для работы с файловым менеджером типа **PurpleFileManager<T>**.

Создать публичные свойства только для чтения, возвращающие текущего менеджера **Manager** типа **PurpleFileManager<T>** и массив задач **Tasks** типа **T**.

Один конструктор должен принимать массив объектов типа **T** и инициализировать внутреннее хранилище, при отсутствии переданных данных должен создаваться пустой массив. Второй конструктор должен принимать менеджер файлов типа **PurpleFileManager<T>** и необязательный массив задач. Третий конструктор должен принимать массив задач и менеджер файлов. Во всех случаях переданный массив должен копироваться во внутреннее хранилище или же должен создаваться пустой массив, если ничего не было передано или был передан **null**.

Создать метод **Add** для добавления одного объекта типа **T** в коллекцию с увеличением размера массива. Создать перегруженный метод **Add** для добавления массива объектов с последовательным добавлением каждого элемента.

Создать метод **Remove**, который удаляет переданный объект из коллекции. Создать метод **Clear**, который очищает коллекцию задач и удаляет директорию, связанную с текущим файловым менеджером через свойство **FolderPath**.

Создать метод **SaveTasks**, который последовательно сохраняет все задачи в файлы с использованием текущего файлового менеджера. Имя файла должно изменяться для каждой задачи с использованием метода **ChangeFileName**, а сохранение должно выполняться с помощью метода **Serialize**.

Создать метод **LoadTasks**, который последовательно загружает задачи из файлов с использованием текущего файлового менеджера с помощью метода **Deserialize** и обновляет элементы массива.

Создать метод **ChangeManager**, который заменяет текущего менеджера на нового, создаёт папку на основе свойства **Name** менеджера (если она отсутствует) и устанавливает её в качестве рабочей папки для нового менеджера с помощью метода **SelectFolder**.

Файл **PurpleFileManager**

Создать абстрактный обобщённый класс **PurpleFileManager<T>**, наследующий класс **MyFileManager** и реализующий интерфейс **ISerializer<T>**, где параметр типа **T** ограничен типом **Lab9.Purple.Purple**.

Конструкторы должны быть идентичны конструкторам базового класса **MyFileManager**.

Переопределить методы **EditFile** и **ChangeFileExtension**, сохранив базовую логику работы с файлом, используя свойство **FullPath** и проверки корректности пути.

Метод **Serialize** должен быть объявлен как абстрактный и принимать объект типа **T**.

Метод **Deserialize** должен быть объявлен как абстрактный и возвращать объект типа **T**.

Файл **PurpleJsonFileManager**

Создать обобщённый класс **PurpleJsonFileManager<T>**, наследующий класс **PurpleFileManager<T>**, где **T** ограничен типом **Lab9.Purple.Purple**.

Конструкторы должны быть идентичны конструкторам базового класса **PurpleFileManager<T>**.

Переопределить метод **EditFile** таким образом, чтобы выполнялась десериализация объекта из файла по пути **FullPath** с помощью метода **Deserialize**, затем вызывался метод **ChangeText** у полученного объекта, после чего объект сериализовывался обратно в файл по тому же пути с помощью метода **Serialize**.

Переопределить метод **ChangeFileExtension** таким образом, чтобы расширение файла изменялось только на «**json**» с использованием метода **ChangeFileFormat**.

Реализовать метод **Serialize** таким образом, чтобы он преобразовывал объект типа **T** в JSON-представление, дополнял его информацией о типе объекта и сохранял результат в файл по пути **FullPath**.

Реализовать метод **Deserialize** таким образом, чтобы он считывал содержимое файла по пути **FullPath**, определял тип объекта на основе сохранённых данных и создавал соответствующий объект типа **T** с последующей инициализацией данных.

Файл **PurpleTxtFileManager**

Создать обобщённый класс **PurpleTxtFileManager<T>**, наследующий класс **PurpleFileManager<T>**, где **T** ограничен типом **Lab9.Purple.Purple**.

Конструкторы должны быть идентичны конструкторам базового класса **PurpleFileManager<T>**.

Переопределить метод **EditFile** таким образом, чтобы выполнялась десериализация объекта из файла по пути **FullPath** с помощью метода **Deserialize**, затем вызывался метод **ChangeText** у объекта, после чего объект сериализовывался обратно в файл с помощью метода **Serialize**.

Переопределить метод **ChangeFileExtension** таким образом, чтобы расширение файла изменялось только на «txt» с использованием метода **ChangeFileFormat**.

Реализовать метод **Serialize** таким образом, чтобы он преобразовывал объект типа **T** в текстовое представление в виде набора пар ключ-значение и сохранял результат в файл по пути **FullPath**. Структура данных должна обеспечивать возможность последующего восстановления объекта при помощи метода **Deserialize**.

Реализовать метод **Deserialize** таким образом, чтобы он считывал содержимое файла по пути **FullPath**, восстанавливал набор данных из текстового представления, определял тип объекта и создавал соответствующий объект типа **T** с последующей инициализацией данных.

Файл **PurpleXmlFileManager**

Создать обобщённый класс **PurpleXmlFileManager<T>**, наследующий класс **PurpleFileManager<T>**, где **T** ограничен типом **Lab9.Purple.Purple**.

Конструкторы должны быть идентичны конструкторам базового класса **PurpleFileManager<T>**.

Переопределить метод **EditFile** таким образом, чтобы выполнялась десериализация объекта из файла по пути **FullPath** с помощью метода **Deserialize**, затем вызывался метод **ChangeText** у объекта, после чего объект сериализовывался обратно в файл с помощью метода **Serialize**.

Переопределить метод **ChangeFileExtension** таким образом, чтобы расширение файла изменялось только на «xml». Перед изменением расширения должна выполняться десериализация текущего объекта, после чего расширение изменяется с помощью метода **ChangeFileFormat**, и объект повторно сериализуется в новый файл.

Реализовать метод **Serialize** таким образом, чтобы он преобразовывал объект типа **T** в промежуточное представление (DTO), содержащее информацию о типе объекта, входных данных и дополнительных полях, и сохранял его в XML-формате с использованием класса **XmlSerializer** в файл по пути **FullPath**.

Реализовать метод **Deserialize** таким образом, чтобы он считывал XML-файл по пути **FullPath**, десериализовывал данные в промежуточный объект, определял тип исходного объекта на основе

сохранённых данных и создавал соответствующий объект типа **T** с последующей инициализацией данных. В случае ошибки десериализации должен возвращаться `null`.

Создать вспомогательный тип **DTOPurple**, предназначенный для хранения сериализуемых данных, включающий имя типа, входные данные и дополнительные параметры, необходимые для восстановления объекта.